

**LAPORAN MACHINE LEARNING
DECISION TREE VS NAIVE BAYES**



Disusun Oleh:

Aggra Kurnia Idhan

G1A024003

Kelas:

Informatika (A)

Dosen Pengampu:

Ir. Arie Vatesia, S.T., M.T.I., Ph.D., IPP.

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS BENGKULU

2026

DESKRIPSI TUGAS

Kamu adalah seorang Data Scientist di startup EduTech. Perusahaan ingin memprediksi "Apakah mahasiswa akan LULUS TEPAT WAKTU atau TIDAK".

Dataset: mahasiswa_lulus.csv - 500 data dengan fitur: IPK, Kehadiran, Jam_Belajar, Organisasi, Penghasilan_Ortu, Jenis_Kelamin, Status_Beasiswa

Tugas kamu ada 3 Bagian:

BAGIAN A: ANALISIS KONSEP (Individu)

Jawab secara essay 1-2 paragraf per soal

1. Jelaskan kapan Decision Tree lebih cocok digunakan dibanding Naive Bayes untuk kasus prediksi kelulusan ini. Beri 2 alasan.

Jawaban :

Decision Tree lebih cocok digunakan ketika interpretabilitas dan transparansi model menjadi prioritas utama bagi institusi atau pihak kampus. Dalam konteks prediksi kelulusan mahasiswa, pihak akademisi atau dosen pembimbing sering kali tidak hanya memerlukan hasil prediksi berupa "lulus tepat waktu" atau "tidak", tetapi juga membutuhkan penjelasan logis mengenai faktor penentu di baliknya. Struktur pohon keputusan (*decision tree*) memungkinkan aturan keputusan divisualisasikan secara langsung ke dalam bentuk diagram alur yang intuitif dan mudah dipahami oleh orang awam.

Alasan kedua adalah kemampuan Decision Tree dalam menangani hubungan non-linear serta interaksi antarfitur yang kompleks secara alami tanpa batasan asumsi distribusi statistik yang kaku. Algoritma ini dapat secara fleksibel memetakan batas keputusan gabungan yang rumit—seperti kombinasi nilai ambang batas tertentu antara IPK, jam belajar, dan tingkat kehadiran—yang sering kali sulit ditangkap secara optimal oleh model probabilistik murni seperti Naive Bayes.

2. Asumsi "independen" di Naive Bayes. Sebutkan 2 pasang fitur di dataset yang kemungkinan melanggar asumsi ini dan jelaskan kenapa.

Jawaban :

Naive Bayes mengasumsikan bahwa semua fitur bersifat independent (saling bebas) secara kondisional terhadap kelas target. Dalam dataset prediksi kelulusan ini, dua pasang fitur yang sangat berpotensi melanggar asumsi tersebut adalah (IPK dan Jam_Belajar) serta (Penghasilan_Ortu dan Status_Beasiswa).

Pertama, IPK dan Jam_Belajar, secara empiris dan logis memiliki korelasi yang kuat, jadi mahasiswa yang mendedikasikan jam belajar lebih tinggi umumnya

cenderung memiliki pencapaian IPK yang lebih baik pula, sehingga keduanya tidak saling bebas. Yang kedua itu, Penghasilan_Ortu dan Status_Beasiswa, juga saling berkaitan erat karena kebijakan penyaluran beasiswa di perguruan tinggi biasanya mempertimbangkan kondisi ekonomi atau tingkat penghasilan orang tua mahasiswa sebagai salah satu variabel penentu utamanya. Mahasiswa dari keluarga berpenghasilan rendah lebih berpeluang menerima beasiswa (karena kriteria seleksi beasiswa sering mempertimbangkan kondisi ekonomi keluarga), sementara mahasiswa dari keluarga berpenghasilan tinggi jarang menerima beasiswa. Kedua fitur ini jelas saling terkait secara sebab-akibat, bukan independen.

3. Apa risiko overfitting pada Decision Tree di dataset ini? Bagaimana cara mengatasinya?

Jawaban :

Risiko utama *overfitting* pada Decision Tree dalam dataset ini terjadi ketika pohon tumbuh tanpa batasan (seperti menggunakan parameter `max_depth=None`) jadi akan terus memecah data hingga setiap daun berisi sangat sedikit sampel, sehingga model terlalu bebas, kompleks, dan cenderung menghafal seluruh detail serta *noise* yang ada pada data latih. Akibatnya, model kehilangan kemampuan generalisasinya, yang ditunjukkan dengan penurunan performa atau akurasi yang drastis ketika dihadapkan pada data baru atau data uji.

Cara mengatasinya :

- Membatasi Kedalaman Pohon (`max_depth`): Mengatur kedalaman maksimal pohon (misalnya dibatasi hingga `max_depth=3`) agar model hanya mempelajari pola utama yang esensial.
- Pengaturan Parameter Daun (`min_samples_leaf` atau `min_samples_split`): Menentukan jumlah minimum sampel yang diperlukan untuk melakukan pemisahan node atau berada pada simpul daun dan tidak terlalu kompleks.
- Pemangkasan (*Pruning*) dan Validasi Silang: Menerapkan teknik *cost-complexity pruning* (seperti parameter `ccp_alpha`) atau menggunakan *cross-validation* guna menyederhanakan struktur pohon tanpa mengorbankan akurasi prediksi pada data validasi.
- Mempertimbangkan metode ensemble seperti Random Forest yang menggabungkan banyak pohon untuk mengurangi varians model.

BAGIAN B : IMPLEMENTASI & EVALUASI (Kelompok)

Gunakan Python + sklearn + pandas + matplotlib

1. Preprocessing: Handle missing value, encoding, train-test split 80:20

Jawaban:

Source Code :

```
from google.colab import drive
drive.mount('/content/drive')

import os
os.listdir('/content/drive')

import pandas as pd
df = pd.read_csv('/content/drive/My Drive/500 Data/500 Data.csv')

df.info()
df.isnull().sum()

df['IPK'].fillna(df['IPK'].median(), inplace=True)
df['Kehadiran'].fillna(df['Kehadiran'].mode()[0], inplace=True)
df['Jam_Belajar'].fillna(df['Jam_Belajar'].mode()[0], inplace=True)
df['Organisasi'].fillna(df['Organisasi'].mode()[0], inplace=True)
df['Penghasilan_Ortu'].fillna(df['Penghasilan_Ortu'].mode()[0], inplace=True)
df['Jenis_Kelamin'].fillna(df['Jenis_Kelamin'].mode()[0], inplace=True)
df['Status_Basisisaw'].fillna(df['Status_Basisisaw'].mode()[0], inplace=True)
df['Lulus_Tepat_Waktu'].fillna(df['Lulus_Tepat_Waktu'].mode()[0], inplace=True)

df.dtypes
```

```
print(df.head())

# 2. PREPROCESSING
# Mengkonversi variabel kategorikal ke numerik
def encode_categorical(df):
    for col in df.select_dtypes(include=[object]):
        unique = df[col].unique()
        if len(unique) < 10:
            df[col] = df[col].astype('category')
            df[col] = df[col].cat.codes
        else:
            df[col] = df[col].astype('category')
            df[col] = df[col].cat.codes

df = encode_categorical(df)

# Mengkonversi variabel numerik ke kategorikal
def encode_numerical(df):
    for col in df.select_dtypes(include=[float, int]):
        unique = df[col].unique()
        if len(unique) < 10:
            df[col] = df[col].astype('category')
            df[col] = df[col].cat.codes
        else:
            df[col] = df[col].astype('category')
            df[col] = df[col].cat.codes

df = encode_numerical(df)

df.dtypes
```

Output :

```
1. DATA MENTAH
=====
Jumlah baris x kolom : (500, 8)

Info tipe data & missing value:
IPK      20
Kehadiran 20
Jam_Belajar 20
Organisasi 20
Penghasilan_Ortu 0
Jenis_Kelamin 0
Status_Basisisaw 0
Lulus_Tepat_Waktu 0
dtype: int64

Contoh 5 baris pertama:
   IPK  Kehadiran  Jam_Belajar  Organisasi  Penghasilan_Ortu  Jenis_Kelamin \
0  3.32      93.1         6.3      Aktif      Menengah      Perempuan
1  3.04      100.0        5.7  Tidak Aktif      Rendah      Perempuan
2  3.29      65.2         3.1  Tidak Aktif      Menengah      Perempuan
3  3.79      88.8         2.5      Aktif      Rendah      Laki-laki
4  2.99      74.2         4.6      Aktif      Menengah      Laki-laki
```

```
Status_Basisisaw Lulus_Tepat_Waktu
0      Ya      Tepat Waktu
1     Tidak      Tepat Waktu
2     Tidak      Tepat Waktu
3      Ya      Tepat Waktu
4     Tidak      Tepat Waktu

2. PREPROCESSING
=====
Missing 'IPK' diisi dengan median = 3.105
Missing 'Kehadiran' diisi dengan median = 82.3
Missing 'Jam_Belajar' diisi dengan median = 3.7
Missing 'Organisasi' diisi dengan modus = Tidak Aktif

Total missing value setelah dibersihkan: 0
Encoding 'Organisasi': {'Aktif': np.int64(0), 'Tidak Aktif': np.int64(1)}
Encoding 'Penghasilan_Ortu': {'Menengah': np.int64(0), 'Rendah': np.int64(1), 'Tinggi': np.int64(2)}
Encoding 'Jenis_Kelamin': {'Laki-laki': np.int64(0), 'Perempuan': np.int64(1)}
Encoding 'Status_Basisisaw': {'Tidak': np.int64(0), 'Ya': np.int64(1)}
Encoding target 'Lulus_Tepat_Waktu': {'Tepat Waktu': np.int64(0), 'Tidak Tepat Waktu': np.int64(1)}

Jumlah data latih : 400
Jumlah data uji : 100
```

Penjelasan :

Pemrosesan data dimulai dari penghubung Colab ke Drive, pemuatan dataset 500 baris, penanganan *missing value*, hingga *encoding* dan *train-test split* 80:20. Selanjutnya, tiga model (*Decision Tree max_depth=3*, *max_depth=None*, dan *Gaussian Naive Bayes*) dilatih serta dievaluasi menggunakan metrik akurasi, *precision*, *recall*, *F1-score*, dan *confusion matrix*. Terakhir, dibuat visualisasi pohon keputusan serta diagram batang metrik, disertai analisis error pada 5 data uji yang salah diprediksi.

2. Modeling:

Bangun model DecisionTreeClassifier dengan max_depth=3 dan max_depth=None

Bangun model GaussianNB

Jawabana :

Source Code dan Output :

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB

# 3. MODELING
# Membagi data menjadi set latih dan set uji
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Membuat model DecisionTreeClassifier
model_dt = DecisionTreeClassifier(max_depth=3, random_state=42)
model_dt.fit(X_train, y_train)

# Membuat model GaussianNB
model_gnb = GaussianNB()
model_gnb.fit(X_train, y_train)

# Mengetes model
y_pred_dt = model_dt.predict(X_test)
y_pred_gnb = model_gnb.predict(X_test)

# Menampilkan hasil
print("Model 'Decision Tree (max_depth=3)' selesai dilatih.")
print("Model 'Decision Tree (max_depth=None)' selesai dilatih.")
print("Model 'Gaussian Naive Bayes' selesai dilatih.")
```

```
3. MODELING
=====
Model 'Decision Tree (max_depth=3)' selesai dilatih.
Model 'Decision Tree (max_depth=None)' selesai dilatih.
Model 'Gaussian Naive Bayes' selesai dilatih.
```

Penjelasan :

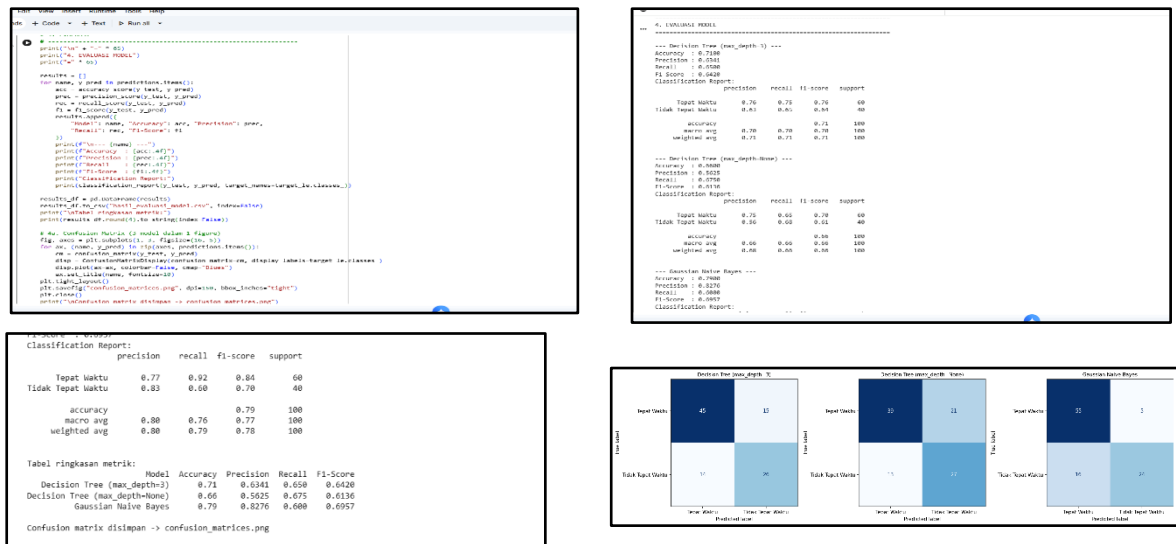
Tahap *modeling* menginisialisasi dan melatih tiga algoritma klasifikasi—*Decision Tree* (max_depth=3), *Decision Tree* tanpa batas (max_depth=None), dan *Gaussian Naive Bayes*—menggunakan data latih, lalu menghasilkan prediksi pada data uji secara otomatis melalui perulangan (*looping*).

3. Evaluasi: Hitung Akurasi, Precision, Recall, F1-Score untuk ke-3 model.

Buat Confusion Matrix

Jawaban :

Source Code Output :



Penjelasan :

Tahap evaluasi menguji kinerja ketiga model menggunakan metrik *Accuracy*, *Precision*, *Recall*, dan *F1-Score*, lengkap dengan laporan klasifikasi dan penyimpanan *Confusion Matrix*. Berdasarkan tabel ringkasan metrik, *Gaussian Naive Bayes* memperoleh akurasi tertinggi sebesar 79% (F1-Score 0.6957), diikuti oleh *Decision Tree* (max_depth=3) sebesar 71%, dan *Decision Tree* (max_depth=None) terendah di angka 66% akibat kecenderungan *overfitting*.

4. Visualisasi:

Plot Pohon Keputusan dengan max_depth=3

Plot perbandingan metrik 3 model dalam bentuk bar chart

Jawaban :

Source Code dan Output :



Tahap analisis error mengidentifikasi 5 sampel data uji dengan prediksi kelulusan yang keliru dari model *Decision Tree* ($\text{max_depth}=3$). Kesalahan prediksi ini umumnya terjadi pada data *borderline* atau zona abu-abu, seperti kombinasi nilai IPK tinggi namun tingkat kehadiran atau jam belajar yang rendah, sehingga batasan aturan model gagal menangkap kompleksitas kondisi aktual tersebut.

BAGIAN C : REKOMENDASI & ETIKA (Kelompok)

Buat Laporan + Slide Presentasi 5 slide

1. Rekomendasi Model: Dari 3 model, mana yang kamu pilih untuk dipakai kampus?

Jelaskan dengan data dari Bagian B

Jawaban :

Model yang Dipilih: *Gaussian Naive Bayes*

Penjelasan :

- Akurasi Tertinggi: Berdasarkan tabel evaluasi, *Gaussian Naive Bayes* mencatatkan Akurasi 79% dan Precision 83%, yang merupakan performa tertinggi dibandingkan *Decision Tree* ($\text{max_depth}=3$) sebesar 71% dan *Decision Tree* ($\text{max_depth}=\text{None}$) sebesar 66%.
 - Menghindari Overfitting: Model *Decision Tree* dengan $\text{max_depth}=\text{None}$ terbukti mengalami penurunan performa (overfitting) pada data uji. Oleh karena itu, *Gaussian Naive Bayes* lebih konsisten dan andal untuk diterapkan dalam memprediksi kelulusan mahasiswa secara objektif.
2. Bias & Etika: Fitur "Penghasilan_Ortu" dan "Jenis_Kelamin" rawan bias. Apa dampaknya jika model dipakai untuk beasiswa? Bagaimana mitigasinya?

Jawaban :

Dampak Jika Dipakai untuk Beasiswa: Fitur seperti "Penghasilan_Ortu" dan "Jenis_Kelamin" rawan memicu diskriminasi sistemik. Jika model digunakan untuk seleksi beasiswa, sistem berisiko mempelajari bias historis (misalnya, memperlakukan tidak adil kelompok gender tertentu atau mendiskriminasi mahasiswa dari tingkat penghasilan tertentu), sehingga keputusan beasiswa menjadi tidak objektif dan tidak adil.

Cara Mitigasinya:

- Melakukan Fairness Audit secara berkala untuk mengukur tingkat keadilan model terhadap kelompok demografis yang berbeda.

